CODERSINPUNE HACKATHON 2026 · ROUND 1: THE PLAN

BATON

Pass the knowledge, not just the code.

Problem statement: Prevent critical knowledge from leaving with people. Build a system that converts scattered project activity into a usable handover when a team member leaves.

TEAM
Mumbai_wale

TRACK
Full Stack Development

MEMBERS
Vikrant GP · Tanmay Khurpe · Krishna Gupta · Komal Chaurasiya

SUBMISSION
Round 1 proposal · 6 November 2026

Contents

1. Understanding of the problem statement
2. Users affected by the problem
3. Proposed solution
4. Complete product workflow
5. Main features
6. Technology stack and technical plan
7. Team responsibilities
8. Development plan across three rounds
9. Expected impact
10. Future scope
11. Team and participant details

How this proposal maps to the judging criteria

Criterion	Points	Where to find it
Understanding of the problem	25	Sections 1 and 2 (with survey evidence)
Proposed solution	25	Sections 3 and 4 (workflow, sample handover, wireframe)
Technical plan	20	Section 6 (architecture, evidence model, challenges)
Originality of approach	15	Section 3, "What makes Baton different"
Feasibility and scope	15	Sections 5 and 8 (R2/R3 tags, scope cuts, fallback)

1 Our understanding of the problem statement

When someone leaves a project, their code stays but their **reasoning goes with them**. The next person loses days rebuilding context, repeats old mistakes, or breaks something they didn't know was connected.

What is lost

- **The reasons behind decisions:** why this library, why this design, why not the alternative.
- **Unfinished work:** half-done branches, open tasks, promises made to others.
- **Dependencies:** which modules, tasks and people rely on each other.
- **Recurring problems:** bugs and workarounds the team has already fought.
- **Important contacts:** who to ask about what.

Why existing documentation fails

1. **It records *what*, not *why*.** A README explains how to run the code, not why it was built that way.
2. **It goes stale.** Docs fall behind the code within weeks.
3. **It isn't linked to evidence.** A doc can't point to the commit or discussion it describes.
4. **Nobody writes it while busy.** Reasoning is freshest when there's no time to record it.
5. **Exits are rushed.** People forget what the next person actually needs.

The part of the problem we will solve

> Most knowledge is lost **before** anyone leaves, because reasoning is never recorded while the work happens. Summarising afterwards can't recover what was never written down. We solve two parts:
>
> 1. **Capture reasoning during the work,** inside the AI coding tools developers already use.
> 2. **Fill the remaining gaps before departure** with a targeted AI exit interview.

Both feed into a handover that is **traceable** (every claim cites a source), **actionable** (clear owners, risks and next steps) and **permission-aware** (people see only what their role allows). A responsible person reviews and approves it before it's shared.

2 The users affected by the problem

Primary setting: small software teams of 2–15 people, such as student clubs, hackathon teams, open-source groups and early-stage startups. Members change often and documentation is weak.

User	Their pain today	What they need from Baton
Admin / team lead	Loses momentum every time someone leaves and has no view of the risk in advance	Start offboarding, then review, correct and approve the handover. See bus-factor risks early.
Leaving member	Must explain months of work in a rushed final week	A short, focused exit interview, plus control over what was captured about their work
Incoming member	Spends days reading code and old chats to rebuild context	A role-filtered, cited handover with clear next steps

Departure scenarios: a club lead graduates mid-project · an intern's term ends · a teammate moves to another project · a hackathon member drops out between rounds.

Evidence: Google Form survey (8 respondents, 6 Nov 2026)

87.5%
(7/8) lost **over two days** understanding a teammate's work after they left

75.0%
(6/8) said the **reasoning behind decisions** was the hardest part to recover

87.5%
(7/8) had **no formal or written handover** process

> *"Their code was there, but nobody knew why they wrote it that way. We wasted days trying to figure it out."*
> Survey respondent

This is a small pilot sample. We will extend the survey and run usability interviews in Rounds 2 and 3.

3 Our proposed solution

Baton is a web platform paired with an **MCP server**. MCP (Model Context Protocol) is an open standard, so **one server works with Claude Code, Cursor, Codex and Antigravity**.

Two capture layers

- **GitHub sync:** the guaranteed baseline. It collects commits, PRs and issues automatically.
- **The MCP agent:** records the reasons behind changes while people code, through tools such as `log_decision`, `log_blocker` and `get_context`.

When someone is leaving, Baton:

1. **Detects gaps:** unmerged branches, TODOs with no explanation, decisions with no recorded reason.
2. **Interviews the leaver** with 5–10 targeted questions. Their answers become verified knowledge.
3. **Generates a handover** where every claim cites its source and is labelled **Verified** **Inferred** **Missing / Conflicting**
4. **Lets the admin review it.** The admin edits, rejects or approves each section.
5. **Shares a role-filtered view.** The incoming person acknowledges receipt.

What makes Baton different

	Generic AI summariser	Baton
When it captures	After the person leaves	**While the work happens**, through MCP
Gaps	Ignored, or filled with guesses	**Detected and asked about** in an exit interview
Trust	AI output presented as fact	**Cited, labelled and approved by a person**
Access	Everyone sees everything	**Role-filtered**, with secrets redacted
Risk	Visible only after someone leaves	**Bus-factor alerts in advance**

> **Our key insight:** a handover tool should do more than summarise. It should know what it *doesn't* know and ask the right person before they leave.

4 The complete product workflow



#	Step	What happens
1	**Setup**	Log in → **Create** or **Join** an organisation → set the project name and roles, connect GitHub → share the invite link. Each org has at least two admins, so approval never depends on one person.
2	**Connect**	Each member pastes one MCP config into their AI coding tool.
3	**Capture**	GitHub sync plus MCP tools record activity. Tasks have discussion threads, assignees and deadlines. Secrets are redacted **before storage**. Members can view, edit or delete their own captured memories.
4	**Monitor**	The dashboard shows each member's contribution summary, which they can see too, and bus-factor alerts such as *"Only one person has touched* `/payments`*."*
5	**Offboard**	The admin clicks **"Start handover for X."** Baton detects gaps and runs the exit interview.
6	**Generate**	The handover covers what's built, what's left, why key decisions were made, a timeline, risks, dependencies, blockers and next actions with owners. Every claim is cited and labelled.
7	**Review**	The admin edits, rejects or approves each section. Every version is saved.
8	**Handover**	The incoming person sees a role-filtered view, asks questions in a Q&A that answers with sources, and clicks **Acknowledge**.

Sample handover output (synthetic example)

Handover: Rahul S. → Priya M. (Backend role) Approved by admin · v3

WHY DECISIONS WERE MADE

Verified Auth moved from Firebase to Supabase because Firebase pricing exceeded the budget → `[PR #42]`, confirmed in the exit interview

UNFINISHED WORK

Inferred Payment webhook is probably incomplete. This is inferred from an unmerged branch with no recent commits → `[branch feat/pay]`

CONFLICTS

Conflicting The launch deadline is 10 Nov in Task #7 but 15 Nov in a thread → `[Task #7] [Msg 213]`. Needs admin confirmation.

RISKS AND DEPENDENCIES

Risk `/payments` has one contributor (bus factor = 1). The email service depends on it → `[commit a91f3c]`

NEXT ACTIONS

1. Finish and test the webhook (owner: Priya, due 12 Nov) · 2. Confirm the launch date (owner: Admin)

2 items hidden (Infra role only) · 3 secrets redacted · Acknowledged by Priya M.

Wireframe: admin review screen



5 The main features we intend to build

R2 = core flow working in Round 2 · R3 = committed Round 3 improvement

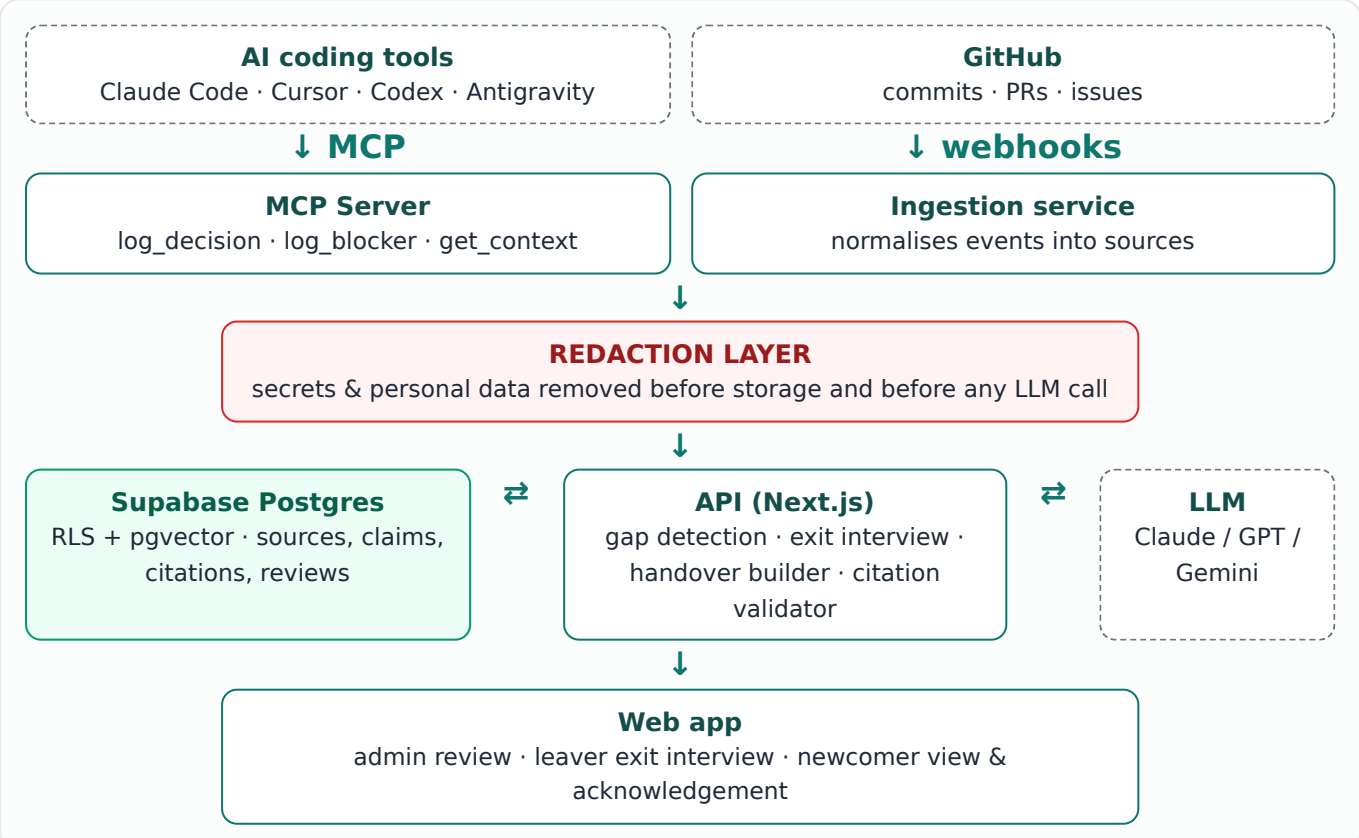
#	Feature	Round
1	Organisation, roles and invite links	R2
2	GitHub ingestion of commits, PRs and issues	R2
3	MCP server with capture tools, working with all major AI coding tools	R2
4	Tasks with threads, assignment and deadlines	R2
5	Redaction of secrets and personal data before storage	R2
6	Handover with citations and confidence labels	R2
7	Review, approval, version history and acknowledgement	R2
8	Gap detection and AI exit interview	R2 basic → R3 ranked by risk
9	Bus-factor risk dashboard	R3
10	Role-based newcomer Q&A with cited answers	R3

Not building during the event: Slack, Notion, Jira or Drive connectors, real-time chat (we'll use task threads instead), and mobile apps.

6 The technology stack we plan to use

Layer	Choice	Why we chose it
Frontend	Next.js, Tailwind CSS, shadcn/ui	A small team can ship a polished UI quickly
Backend and database	Supabase (Postgres, Auth, Row-Level Security)	Permissions are enforced **in the database**, not just hidden in the UI
Memory search	pgvector	Memories and sources live in one database, so a citation is a simple join
Agent layer	MCP TypeScript SDK, LLM API (Claude, GPT or Gemini)	One open standard covers four AI coding tools
Integrations	GitHub App with webhooks	Reliable, permission-scoped access to repository activity
Redaction	Regex rules in the style of Gitleaks, plus an entropy check	Proven patterns for detecting secrets
Testing	Vitest and Playwright	Unit tests plus end-to-end tests of the handover flow
Hosting	Vercel + Supabase Cloud	Free tiers and fast deployment for judges

System architecture



Evidence model

```
Source   (id, type: commit|pr|issue|task|thread|memory|interview, url, author, timestamp,
visibility)
Claim    (id, handover_id, section, text, confidence: verified|inferred|missing|conflicting)
Citation (claim_id, source_id)
Review   (claim_id, reviewer, action: edit|approve|reject, version, timestamp)
```

Label	Rule
Verified	A cited source directly supports the claim, or the leaving person confirmed it in the exit interview.
Inferred	The claim is derived from indirect signals, such as a stale branch.
Missing	The claim has no valid source.
Conflicting	Two sources disagree. Both are shown and nothing is silently picked.

Main technical challenges and how we will handle them

1. **Accurate citations (the hardest part).** The LLM only receives retrieved sources and must cite their IDs. A validator rejects any citation that isn't in the retrieved set. Claims without a valid citation are automatically downgraded to Inferred or Missing, so a guess is never presented as fact.
2. **Redaction without losing context.** Detection runs **before storage and before any LLM call**. Each secret becomes a labelled placeholder such as `[REDACTED: API_KEY]`, so the surrounding context survives.
3. **Permission-aware views.** Every source has a visibility level. Row-Level Security filters data at query time and during generation, so hidden data never reaches the LLM when it builds that viewer's handover.
4. **Reliable capture.** AI agents won't always call our tools, so GitHub sync is the guaranteed baseline and MCP adds the reasoning on top.

7 The responsibilities of each team member

Member	Role	Responsibilities
Vikrant GP	Frontend & Design Lead	Design system, org setup and invite flow, dashboard, task threads, handover review UI, newcomer view
Tanmay Khurpe	Backend Developer	Database schema, auth, Row-Level Security, GitHub App and webhooks, MCP server, citation validator, version history
Krishna Gupta	UI/UX Designer & Business Analyst	User research and survey, wireframes and user flows, exit-interview question design, handover format and AI prompt design
Komal Chaurasiya	QA & User Experience Lead	Test plan, redaction and permission tests, synthetic demo data, usability testing, documentation

8 Our development plan across all three rounds

Round	Goal	Deliverables
Round 1: Plan	Understand the problem and design the solution	Research, survey, architecture, evidence model, wireframes, this proposal
Round 2: Build	The core flow works end to end	Every `R2` feature: setup → GitHub and MCP capture → redaction → cited, labelled handover → review and approve → acknowledge, all on synthetic data
Round 3: Finish	Deliver three committed improvements	1) A full exit interview that ranks gaps by risk 2) A bus-factor dashboard 3) Newcomer Q&A with cited answers. Plus testing, accessibility and documentation.

Why this is feasible

- **A focused scope:** one real integration (GitHub) plus in-app tasks. Everything else is future scope.
- **No unrealistic access:** we use a **synthetic demo repository** with fake contributors, commits, PRs, issues, unfinished branches and **planted fake secrets** to prove redaction works. No confidential data is used.
- **A fallback:** if MCP takes longer than planned, GitHub sync plus task threads still deliver the full core flow.
- **A clear split:** each of the four members owns a separate part of the product, so work runs in parallel.

9 The expected impact of our solution

- **Faster onboarding:** new members start from a cited, structured handover instead of reading code and chats from scratch. In our survey, 7 of 8 people lost more than two days doing that.
- **Less repeated work:** past decisions and recurring problems stay visible, so they aren't relived.

- **Risk visible early:** bus-factor alerts show which knowledge sits with only one person, *before* that person leaves.
- **Trustworthy AI:** every claim is cited, labelled and approved by a person, so the handover never passes off guesses as facts.
- **Safe sharing:** secrets are redacted and each role sees only what it's allowed to.

10 The future scope of the product after the event

- Slack, Notion, Jira and Google Drive connectors
- Support for non-technical teams such as HR, marketing and research groups
- Ingesting meeting transcripts
- A continuously updated "living handover" for every member
- Self-hosting for organisations with strict data-residency rules
- Organisation-wide knowledge-risk analytics

11 Complete team and participant details

Team name: Mumbai_wale

#	Full name	Role	Email	Phone
1	**Vikrant GP**	Frontend & Design Lead	gpvikrant8@gmail.com	+91 73042 43682
2	**Tanmay Khurpe**	Backend Developer	may.911.505@gmail.com	+91 84549 25469
3	**Krishna Gupta**	UI/UX Designer & Business Analyst	krishna2205gupta@gmail.com	+91 98200 83134
4	**Komal Chaurasiya**	QA & User Experience Lead	komalchaurasiya191@gmail.com	+91 89289 44524

What each member will personally build

- **Vikrant GP (Frontend & Design Lead):** "I'm owning the user interface and frontend experience end-to-end. I'll be building out the organization setup and invite flows, the team dashboard, and the task discussion threads. My main focus will be the interactive handover review screen where team leads can inspect citation links, approve or tweak sections, and the clean newcomer view where incoming members review their handover and acknowledge receipt."
- **Tanmay Khurpe (Backend Developer):** "I'm handling the backend architecture, data pipelines, and agent integration end-to-end. I'll design our Supabase Postgres schema with Row-Level Security and pgvector so role-based handovers are strictly enforced at the database level. On the agentic side, I'll build the MCP server using the TypeScript SDK to expose capture tools like log_decision and log_blocker directly inside IDE tools like Cursor, Claude Code, Antigravity, and Codex. I'm also implementing the GitHub webhook ingestion pipeline, the Gitleaks-style entropy redaction layer, and the citation validator that cross-checks LLM claims against raw source IDs so unsupported or hallucinated claims are flagged for review instead of being presented as fact."
- **Krishna Gupta (UI/UX Designer & Business Analyst):** "I'm in charge of how the product looks and how it feels to use. I start by understanding what the people on each side of a handover actually need, then map out the full flow and design wireframes for the key screens. My main goal is to make handing over work feel easy, whether you're the person leaving or the person taking over. I also work closely with our frontend developer so what we build matches what we designed."
- **Komal Chaurasiya (QA & User Experience Lead):** "I will be responsible for preparing the test plan for BATON and ensuring that its features work correctly. I will conduct redaction and permission testing to verify that confidential information is protected and users can access only the information permitted for their roles. I will prepare synthetic demo data for testing and demonstrations, perform usability testing to ensure the platform is easy to use, and maintain proper project and testing documentation."

Disclosures

- No code existed before the hackathon.
- All demo data will be synthetic.
- AI assistance (Claude) was used for planning and drafting this proposal.
- Open-source libraries (Next.js, Supabase, MCP SDK) will be used under their licences.